AUDIT TOOLKIT LAB VALIDATION & TESTING GUIDE

Controlled Heuristic Verification, Benign Artifact Simulation & Unit Testing Field Manual

Target Scripts: windows_compromise_audit.ps1, server_webshell_rootkit_audit.sh, mobile_forensics_audit.sh	**Lab Paradigm:** Synthetic Benign Artifacts / Atomic Red Team Methodology
Target Lab Environments: Hyper-V / VirtualBox VM, Linux VPS Sandbox, Android Emulator / Test Phone	**Remediation Portal:** http://127.0.0.1:8888 (Auto-Spawned & Verified)

[ALERT] GOLDEN RULE OF DETECTION TESTING: ISOLATED LABS & CONTROLLED SIMULATION

In professional security engineering and detection QA, security tools are NEVER tested by intentionally infecting production machines or live personal hardware with live malware. Instead, practitioners utilize **Atomic Adversary Emulation**—creating inert, synthetic, benign mock artifacts (e.g. harmless dummy registry keys, inert test scripts with eval wrappers, and non-malicious mock services) inside an isolated Virtual Machine (VM). This proves detection efficacy with 100% mathematical certainty while preserving zero risk of host destabilization.

1. Lab Architecture & Safety Prerequisites

To rigorously validate that every [ALERT] and [WARN] trigger functions as designed, establish the following isolated testing topologies:

- **Windows VM Lab:** Deploy a Windows 10/11 Enterprise Evaluation or Windows Server 2022 virtual machine (Hyper-V, VirtualBox, or VMware Workstation). Configure the network adapter to **Host-Only** or **Internal**. Take a baseline snapshot (e.g., Clean_Baseline_Snapshot) prior to test execution.
- **Linux Server Sandbox:** Set up a local test directory (e.g. /tmp/audit_test_lab/) on your Kali Linux VM or a dedicated disposable cloud VPS ($5 droplet). No root-level host destruction is required; all simulated webshells and `.htaccess` rules can be placed inside an isolated mock webroot.
- **Mobile Testing Lab:** Use an **Android Virtual Device (AVD)** via Android Studio or a dedicated, factory-reset physical Android test phone connected over USB with Developer Options enabled.

2. Validating windows_compromise_audit.ps1 (Controlled Emulation)

The following PowerShell procedures plant harmless synthetic triggers in the test environment. Running windows_compromise_audit.ps1 will detect each synthetic anomaly, proving the detection logic works.

Step 2.1: Plant Benign Synthetic Test Triggers (Windows PowerShell Admin):

Run the following benign simulation script in an Administrator PowerShell window on your test VM:

```
# ============================================================================
# STEP 1: SIMULATE TEST ANOMALIES (BENIGN MOCK TRIGGERS)
# ============================================================================

# Trigger A: Unsigned Mock Executable in User Temp Directory
$testDir = Join-Path $env:TEMP 'SimulatedThreatLab'
New-Item -ItemType Directory -Path $testDir -Force | Out-Null
$mockExe = Join-Path $testDir 'mock_unsigned_beacon.exe'
# Copy standard cmd.exe as an unsigned mock payload to Temp
Copy-Item (Get-Command cmd.exe).Source -Destination $mockExe -Force
$testProc = Start-Process -FilePath $mockExe -ArgumentList '/k timeout /t 300' -PassThru
Write-Host "[+] Spawned mock process: PID $($testProc.Id) from $mockExe" -ForegroundColor Green

# Trigger B: Registry Run Key LOLBin Persistence
$regPath = 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'
Set-ItemProperty -Path $regPath -Name 'Simulated_Backdoor_Trigger' -Value 'powershell.exe -enc V3JpdGUtSG9zdCAiVW5pdFRlc3Qi'

# Trigger C: IFEO Debugger Hijack Mock
$ifeoKey = 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\mock_target_app.exe'
New-Item -Path $ifeoKey -Force | Out-Null
Set-ItemProperty -Path $ifeoKey -Name 'Debugger' -Value 'cmd.exe'

# Trigger D: Non-Microsoft Scheduled Task with LOLBin & Encoded Flags
$action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument '-enc V3JpdGUtT3V0cHV0IFRlc3Q= -Bypass'
$trigger = New-ScheduledTaskTrigger -AtLogon
Register-ScheduledTask -TaskName 'Simulated_APT_Task' -Action $action -Trigger $trigger -Description 'Audit QA Test' -Force
| Out-Null

# Trigger E: Temporary Windows Defender Exclusion
Add-MpPreference -ExclusionPath $testDir -ErrorAction SilentlyContinue
```

Step 2.2: Execute the Windows Auditor:

```
# Execute the production auditor against the test machine:
powershell.exe -ExecutionPolicy Bypass -NoProfile -File .\windows_compromise_audit.ps1
```

Step 2.3: Verification Checklist & Expected Outputs:

Examine the terminal and the generated Compromise_Assessment_Report.txt. Verify the following detections appear:

Target Synthetic Trigger	Module Audited	Expected Report Output Signal	Status
mock_unsigned_beacon.exe in Temp	Module 2: Unsigned Processes	`[-] CRITICAL: Unsigned process executing from user-writable directory (PID: ...)`	**PASS [ALERT]**
Run Key with powershell.exe -enc	Module 4: Registry Run Keys	`[-] Suspicious Registry Persistence entry invoking LOLBin: Simulated_Backdoor_Trigger`	**PASS [ALERT]**
IFEO Debugger on mock_target_app.exe	Module 4: IFEO Debugger Hijacks	`[-] IFEO Debugger Backdoor detected on 'mock_target_app.exe'! Execution intercepted by: cmd.exe`	**PASS [ALERT]**
Task Simulated_APT_Task with -enc	Module 6: Scheduled Tasks	`[-] Suspicious non-Microsoft Scheduled Task detected: Simulated_APT_Task`	**PASS [ALERT]**
Exclusion on $testDir	Module 7: Defender Exclusions	`[-] Defender Path Exclusion Configured (Potential Blindspot): ...\SimulatedThreatLab`	**PASS [ALERT]**

Step 2.4: Immediate Cleanup & Teardown Script:

```
# ================================================================================
# TEARDOWN SCRIPT: PURGE ALL BENIGN TEST ARTIFACTS
# ================================================================================
Stop-Process -Id $testProc.Id -Force -ErrorAction SilentlyContinue
Remove-Item -Path $testDir -Recurse -Force -ErrorAction SilentlyContinue
Remove-ItemProperty -Path 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run' -Name 'Simulated_Backdoor_Trigger' -ErrorAct
ion SilentlyContinue
Remove-Item -Path 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\mock_target_app.exe' -Rec
urse -Force -ErrorAction SilentlyContinue
Unregister-ScheduledTask -TaskName 'Simulated_APT_Task' -Confirm:$false -ErrorAction SilentlyContinue
Remove-MpPreference -ExclusionPath $testDir -ErrorAction SilentlyContinue
Write-Host '[+] All test artifacts eradicated. Baseline state fully restored.' -ForegroundColor Green
```

Step 2.5: Verifying Portal Auto-Spawn & Report Ingestion:

1. Confirm that portal_server.py was automatically spawned in the background by the audit script.
2. Open **http://127.0.0.1:8888** in your browser and verify that the scan win_audit_... appears in the sidebar.
3. Inspect the dynamically generated 4-phase remediation steps (Process termination, Carving, Deletion, Verification) for each detected anomaly.
4. Check the 'Resolved' boxes on the findings to confirm the Host Risk Score drops from 100% to 0%.

3. Validating server_webshell_rootkit_audit.sh (Linux Sandbox)

To test the server webshell and rootkit auditor on Kali Linux or a test Linux VPS, create an isolated synthetic webroot directory populated with harmless inert test files that mimic webshell signatures.

Step 3.1: Construct the Synthetic Webroot Sandbox (Bash):

```
# Create an isolated sandbox directory in /tmp
MOCK_WEBROOT="/tmp/audit_test_sandbox"
rm -rf "$MOCK_WEBROOT"
mkdir -p "$MOCK_WEBROOT/wp-content/uploads/2026/01"
mkdir -p "$MOCK_WEBROOT/includes"

# 1. Create a benign mock PHP file containing obfuscated eval signature
# (This executes a harmless 'echo unit test passed', triggering the signature scanner)
echo '' > "$MOCK_WEBROOT/includes/mock_eval.php"

# 2. Place an inert rogue PHP script inside the upload folder
echo '' > "$MOCK_WEBROOT/wp-content/uploads/2026/01/avatar.php"

# 3. Create a malicious .htaccess file with auto_prepend_file
echo 'php_value auto_prepend_file /tmp/backdoor.php' > "$MOCK_WEBROOT/.htaccess"

# 4. Create a world-writable test configuration file
echo 'DB_PASSWORD=secret_mock_password' > "$MOCK_WEBROOT/.env"
chmod 777 "$MOCK_WEBROOT/.env"

echo "[+] Synthetic mock webroot constructed at $MOCK_WEBROOT"
```

Step 3.2: Execute the Linux Auditor:

```
chmod +x server_webshell_rootkit_audit.sh
./server_webshell_rootkit_audit.sh /tmp/audit_test_sandbox
```

Step 3.3: Verification Checklist & Expected Outputs:

Target Synthetic Trigger	Module Audited	Expected Log Output Signal	Status
mock_eval.php	Module 1: Signature Scanner	`[-] Potential PHP Webshell / RCE primitives detected! eval(base64_decode...`	**PASS [ALERT]**
avatar.php in uploads	Module 2: Upload Directory	`[-] HIGH RISK: Found 1 executable script(s) in static/upload directories`	**PASS [ALERT]**
auto_prepend_file in .htaccess	Module 4: .htaccess Hijacking	`[-] CRITICAL: Suspicious .htaccess directives detected: auto_prepend_file`	**PASS [ALERT]**
World-writable .env file	Module 8: File Permissions	`[-] Found 1 world-writable executable/config file(s): .../.env`	**PASS [ALERT]**

Step 3.4: Linux Cleanup Command:

```
rm -rf /tmp/audit_test_sandbox
echo '[+] Test sandbox deleted cleanly.'
```

Step 3.5: Verifying Linux Report Ingestion into Portal:

1. Confirm that server_security_audit_<TIMESTAMP>.json was generated alongside the plain text log.
2. If testing locally, verify the script transmitted the JSON report to http://127.0.0.1:8888/api/upload_report.
3. If testing on a remote VPS, drag and drop the JSON report or plain text log directly into the Portal dropzone to view the remediation guide.

4. Validating mobile_forensics_audit.sh (Android & iOS)

To test mobile auditing capabilities without risking a production smartphone, utilize an Android Emulator (AVD) or a dedicated test smartphone with standard testing conditions.

Step 4.1: Android Lab Simulation Setup:

1. Start an Android Virtual Device via Android Studio or connect a test phone with USB Debugging enabled.
2. Sideload any standard open-source APK (e.g. F-Droid, VLC, or a sample Hello-World APK) via ADB. Because it was installed via ADB and not Google Play, its installer package will be empty or non-standard.
3. (Optional) Install a test self-signed user Root CA certificate via Settings -> Security -> Install from Storage.

```
# Download and sideload a harmless open-source APK to test sideload detection:
curl -L -o /tmp/sample_test_app.apk https://f-droid.org/F-Droid.apk
adb install /tmp/sample_test_app.apk
```

Step 4.2: Execute the Mobile Auditor:

```
chmod +x mobile_forensics_audit.sh
./mobile_forensics_audit.sh android
```

Step 4.3: Expected Mobile Report Outputs:

- **Verified Boot Check:** Reports [OK] if locked, or [ALERT] if running on an unlocked emulator bootloader.
- **Sideloaded Package Detection:** Flags [WARN] Sideloaded / Non-Play-Store Package: org.fdroid.fdroid (Installer: NONE/Direct-APK).
- **MVT STIX2 IOC Engine:** Updates IOCs from Amnesty International and scans system records without crashing.
- **Cleanup:** adb uninstall org.fdroid.fdroid.

Step 4.4: Verifying Mobile Forensics Ingestion into Portal:

1. Confirm that mobile_forensics_report.json was created in $OUT_DIR.
2. Open **http://127.0.0.1:8888** and select the mobile scan from the sidebar to review the step-by-step uninstallation and root CA removal guides.

5. Validating the Cyber Defense Portal (portal_server.py)

The portal operates as an independent, multi-threaded incident response platform. Follow these steps to validate its standalone capabilities:

Step 5.1: Health Check & Standalone Startup:

```
# Launch the portal manually:
python portal_server.py

# In another terminal, verify the REST health probe:
curl http://127.0.0.1:8888/api/health
# Expected response: {"status": "ok", "version": "1.0.0", ...}
```

Step 5.2: Verifying Pre-Seeded Baseline Reports:

Open **http://127.0.0.1:8888** in your browser. Confirm that the portal immediately loads with three rich baseline reports in the sidebar:

- **sample_windows_audit:** Displays 2 Critical Alerts (Unsigned process in Temp, Registry Run LOLBin) and 1 Warning.
- **sample_linux_vps_audit:** Displays 3 Critical Alerts (PHP webshell with eval, upload directory script, .htaccess hijack).
- **sample_mobile_forensic_audit:** Displays Root CA TLS interception alert and sideloaded APK warning.

Step 5.3: Testing Log File Upload & 'What Went Well / What Went Wrong' Analysis:

Upload any saved console_execution.log, Compromise_Assessment_Report.txt, or .json report using either the top navigation button or central dropzone. Confirm that:

1. **Executive Posture Breakdown:** The portal parses the log and generates the side-by-side **'What Went Well'** (verified clean baselines) and **'What Went Wrong'** (threats, blindspots, script runtime warnings) cards.
2. **Tactical Remediation Mapping:** Each identified issue links directly to its 4-phase copyable fix commands.
3. **Embedded Console Viewer:** The collapsible 'RAW CONSOLE EXECUTION LOG STREAM' card displays verbatim terminal output.

6. Quality Assurance Scorecard & Sign-Off Matrix

Toolkit Component	Lab Verification Target	Test Procedure	Expected Output	Operator Sign-Off
windows_compromise_audit.ps1	Unsigned User-Path Binary	Spawn mock cmd.exe in %TEMP%	`[ALERT]`	**[ ] VERIFIED**
windows_compromise_audit.ps1	Registry Run LOLBin	Set HKCU Run with -enc flag	`[ALERT]`	**[ ] VERIFIED**
windows_compromise_audit.ps1	IFEO Debugger Hook	Set Debugger value on mock key	`[ALERT]`	**[ ] VERIFIED**
server_webshell_rootkit_audit.sh	PHP Obfuscation Regex	eval(base64_decode(...)) mock	`[ALERT]`	**[ ] VERIFIED**
server_webshell_rootkit_audit.sh	Upload Directory Isolation	Place .php in /uploads/ mock	`[ALERT]`	**[ ] VERIFIED**
mobile_forensics_audit.sh	Sideloaded Package Audit	Direct ADB install of sample APK	`[WARN]`	**[ ] VERIFIED**
portal_server.py	Auto-Spawn Mechanism	Run audit script with portal offline	`AUTO-START`	**[ ] VERIFIED**
portal_server.py	Structured JSON Ingestion	POST Compromise_Report.json	`200 SUCCESS`	**[ ] VERIFIED**
portal_server.py	Dynamic Host Risk Meter	Toggle 'Resolved' checkboxes	`100% -> 0%`	**[ ] VERIFIED**

[SUCCESS] LAB VALIDATION COMPLETE: OPERATIONAL READINESS CONFIRMED

By validating each synthetic trigger against the scorecard, operators establish rigorous scientific proof that the detection heuristics, regex parsers, and threat intelligence feeds perform flawlessly in enterprise production.